



Insurance Broker Management System

J2EE Web Components – ITE 5332 – 0NA

Assignment – 3

Hitesh Kamalkant Jha – N01610330

Neel Kachhia – N01609700

Abdul Mubeen Mohammed – N01611677

Prof. Vitalii Bohudskyi

Table of Contents

Project Description	1
Overview	1
Features	1
Design Decisions	1
Architecture	1
File Storage Format	2
Repository Design Pattern	2
Screenshots	4
UML Diagrams.....	15
Collaboration Details	19

Project Description

Overview

The Insurance Broker Management System (IBMS) is designed to ensure that the brokers remain organized while efficiently managing the data pertaining to the consumers and the insurance policies offered to them in a systematically orderly manner. The system provides data persistence using Hibernate and allows creation, updating, and deletion of consumers and policies. The system improves both maintainability and scalability by ensuring the application of the Repository Design Pattern where there exists a clear separation of concerns between the business components and data components.

The system allows brokers to handle activities like customer registration, policy handling, and organization of insurance records which burnish operation management along with ensuring the data consistency and integrity.

Features

- Customer Management:
 - Add, update, delete, and view customer records.
 - Comprehensive customer profiles that store important information.
- Policy Management:
 - Create, edit, and assign insurance policies to customers.
 - Delete outdated policies and manage ongoing ones.
- Data Persistence:
 - Store customer and policy data in SQL for easy management and durability.
- Claims Management:
 - Submit, update, and view insurance claims.
 - Filter claims by date range, status, and policy ID.
 - Retrieve all claims or specific ones with detailed summaries for effective claim resolution.
- Policy Cancellation:
 - Brokers can cancel active policies and log the cancellation reason.

- Prevent accidental cancellation of expired or invalid policies.

Design Decisions

System Architecture

The system is structured with a modular approach to simplify the management of customers, insurance policies and future extensions. The key layers of the system are:

Model Layer:

- Classes like Customer and Policy represent the domain entities of Insurance Broker Management System.

View Layer:

- Developed using Nord Design library, this layer provides simple yet modern interface for brokers to manage customers and insurance policies.

Controller Layer:

- This handles the HTTP requests from the client and interacts with the repository layer to fetch or manipulate data.

Repository Layer (Data Access Layer):

- This is responsible for accessing the data storage. In this case, it's interacting with JSON files for reading and writing customer or policy data.

Repository Design Pattern

- The Repository pattern is used in application to abstract and encapsulate the logic for accessing data sources. It provides a way to separate the data access logic from the business logic for loose coupling.

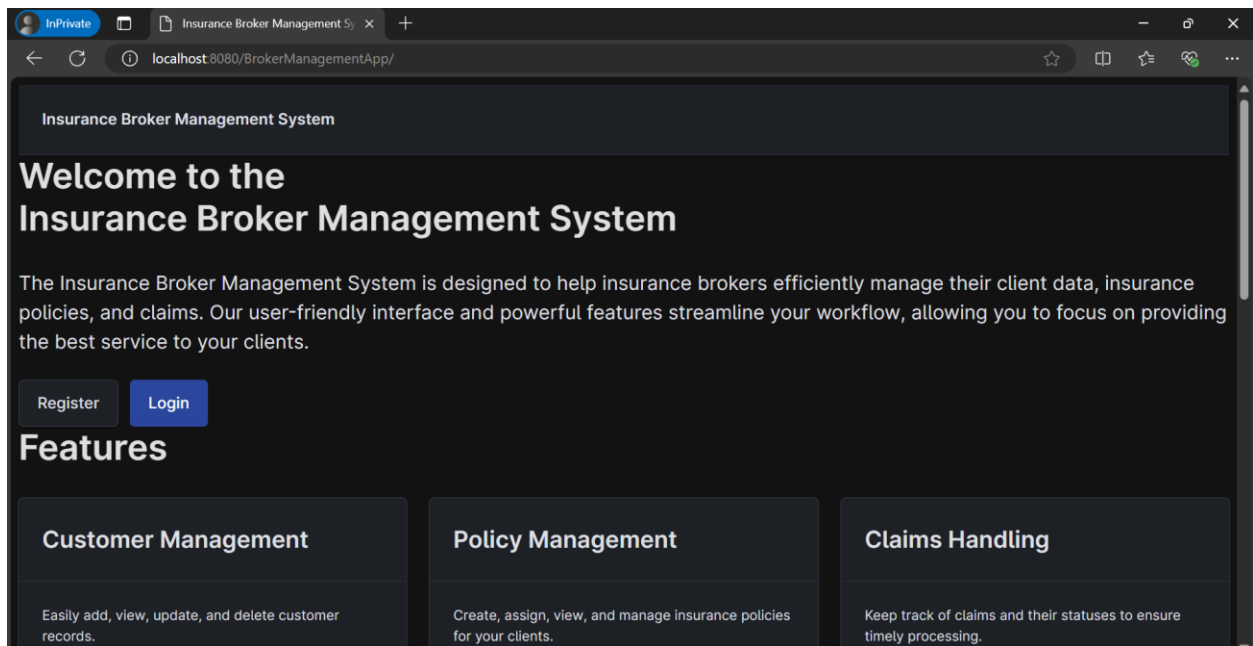
- The Repository Design Pattern provides a clear abstraction for data access, making the system flexible and maintainable. This architecture could be extended to use databases or other storage mechanisms in the future, without needing to change the business logic.

Hibernate Integration

- Hibernate ORM is utilized to map domain entities to database tables.
- Example mappings include:
 - Policy entity to the policy table.
 - Claim entity to the claim table with relationships.
- Configuration ensures the system supports advanced queries and dynamic filtering using Hibernate's Criteria API and HQL.

Screenshots

Welcome Page



Login

Log In as a Broker

Username

enter your email address

Password

enter your password

Log In

New User? [Register](#).

Register

Register as a Broker

Username

enter your username

Email Address

enter your email address

Password

enter your password

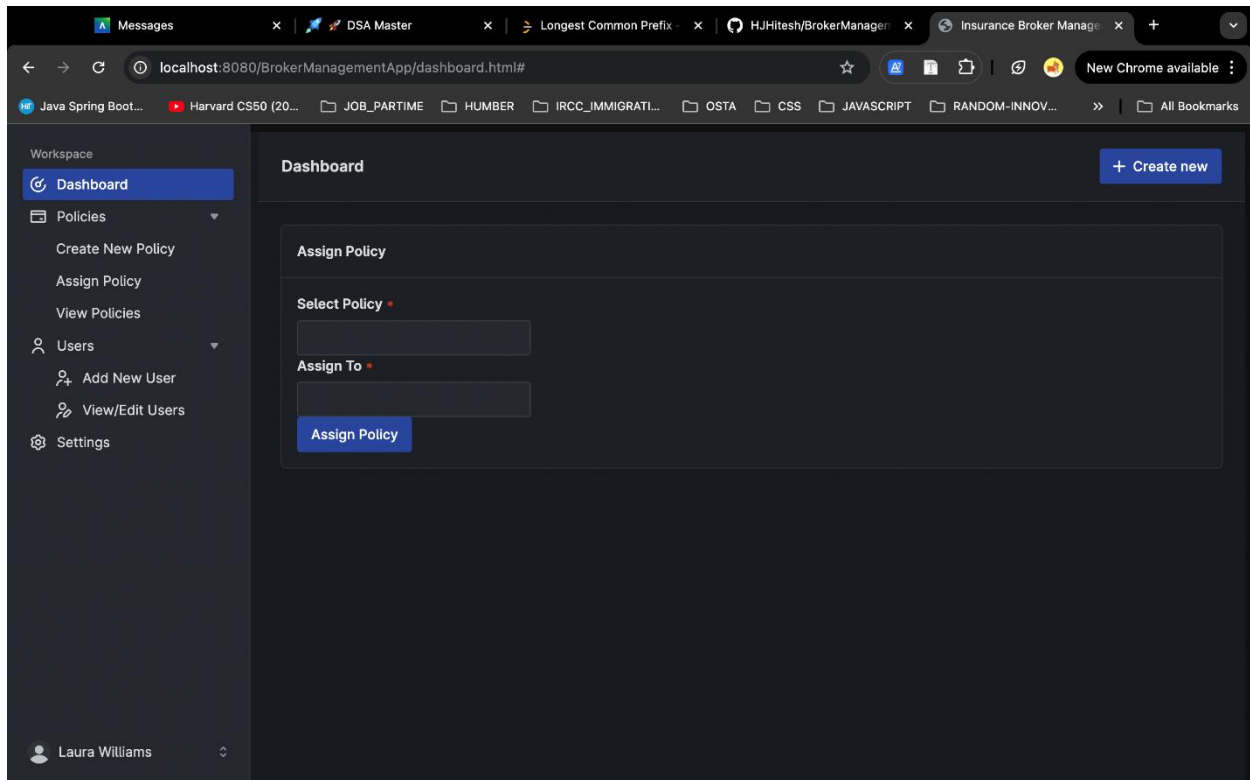
Confirm Password

confirm your password

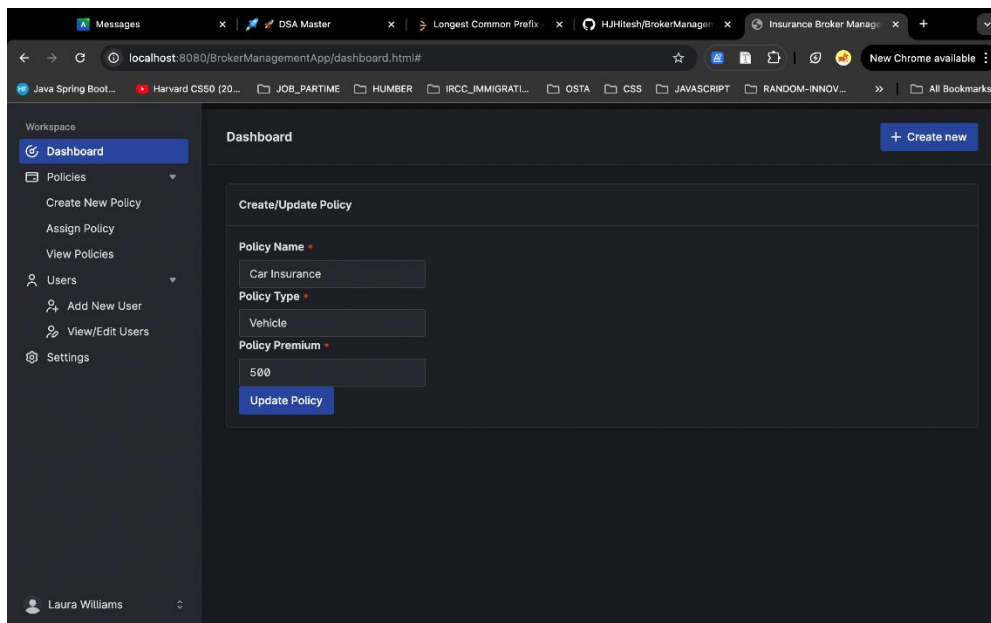
Register

Already an user? [Login](#).

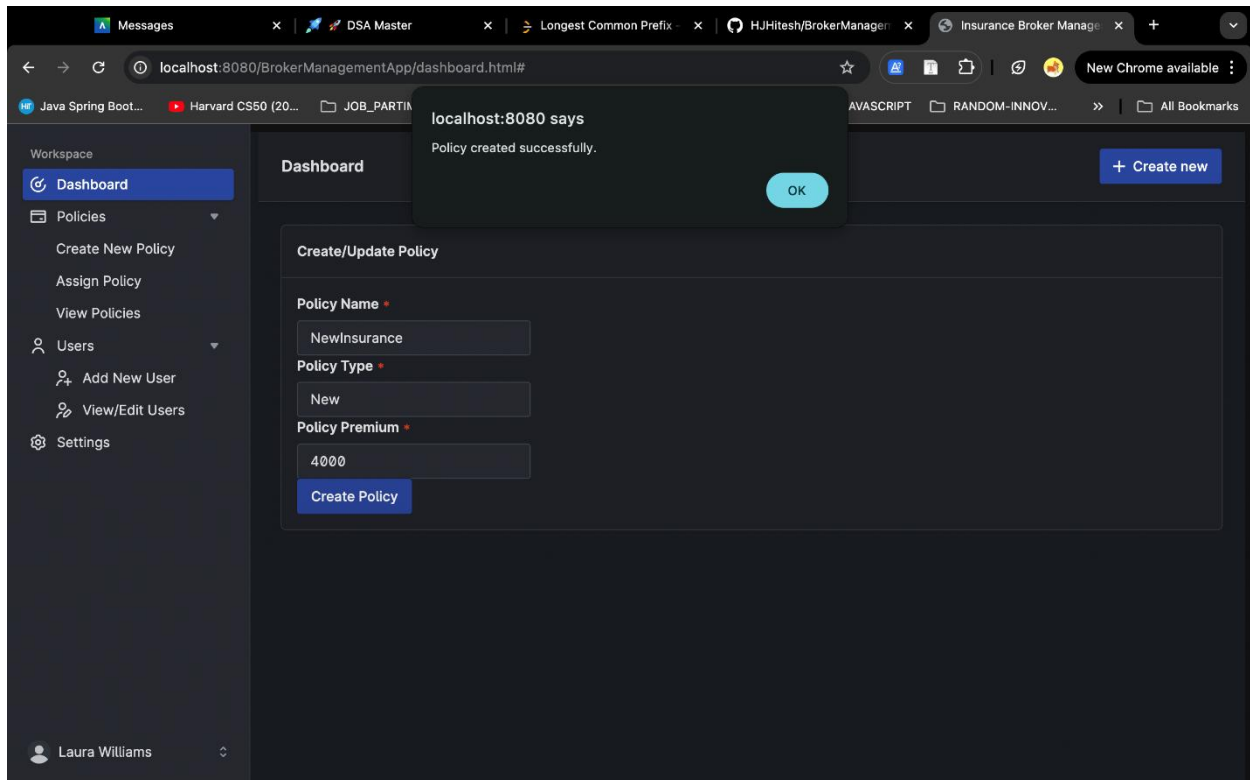
Assign policy



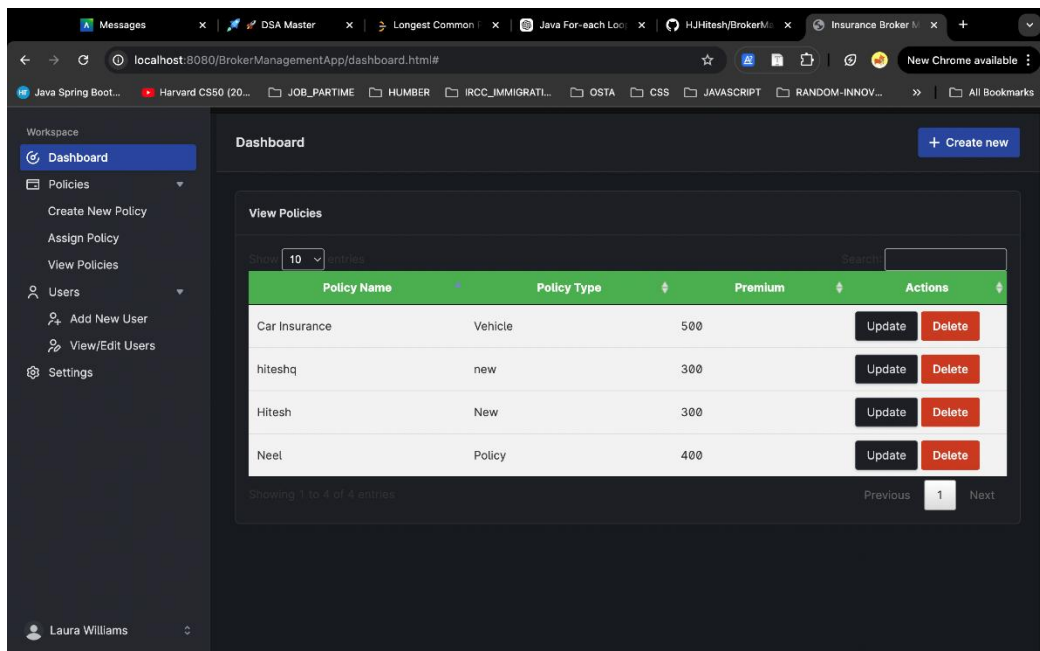
Create policy



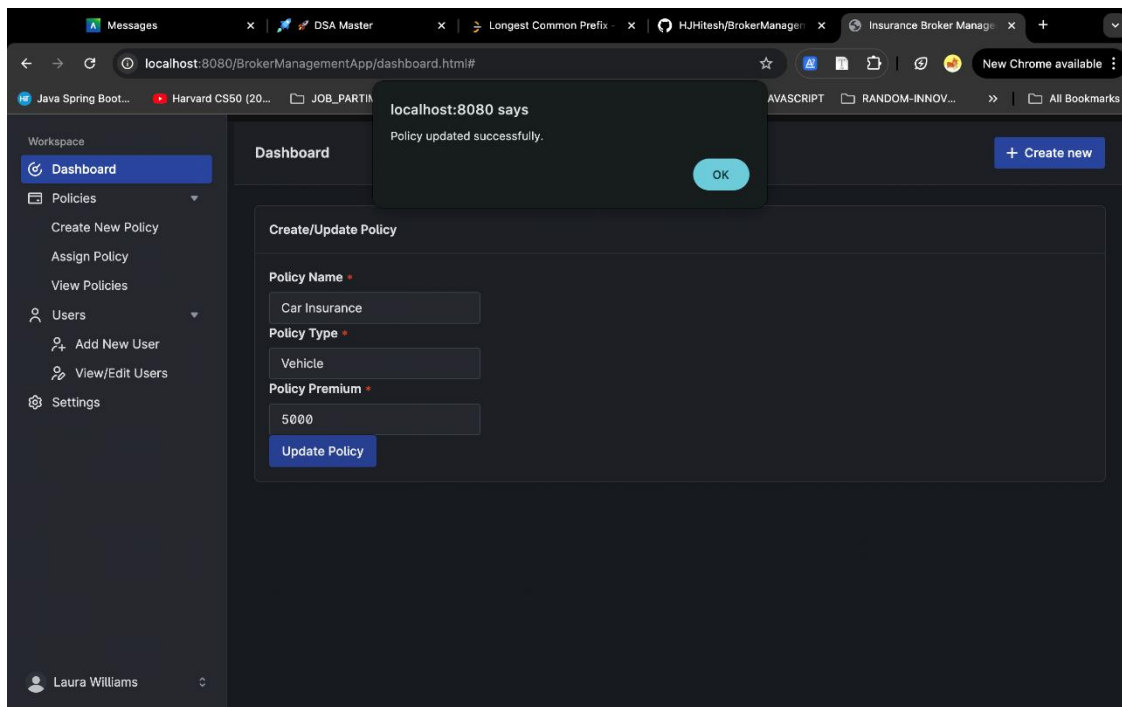
Policy create success



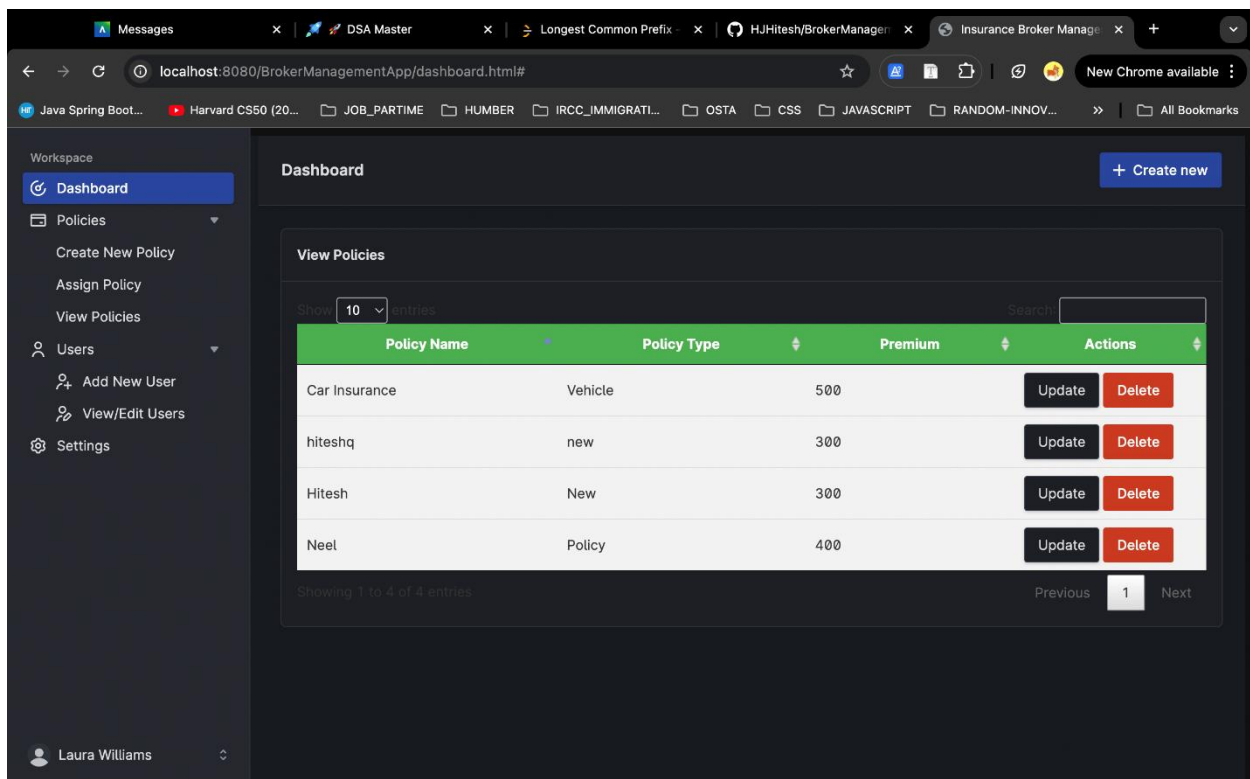
List Policy Delete



Update policy



View policy



Add new user

The screenshot shows the 'Add New User' form in the Insurance Broker Management App dashboard. The form is located in the main content area, titled 'Add New User'. It contains three input fields: 'Username' with the value 'Riya', 'Email' with the value 'riya@gmail.com', and 'Phone Number' with the value '66666666'. Below the input fields is a blue 'Add User' button. The dashboard sidebar on the left shows the 'Users' menu expanded, with 'Add New User' selected. The top navigation bar includes a 'Create new' button.

Dashboard

+ Create new

Add New User

Username *

Riya

Email *

riya@gmail.com

Phone Number

66666666

Add User

View User

The screenshot shows the 'View/Edit Users' table in the Insurance Broker Management App dashboard. The table is located in the main content area, titled 'View/Edit Users'. It displays a list of users with columns for Name, Email, Phone, and Actions. The table has a search bar and a 'Show 10 entries' dropdown. The 'Actions' column contains 'Edit' and 'Delete' buttons for each user. The dashboard sidebar on the left shows the 'Users' menu expanded, with 'View/Edit Users' selected. The top navigation bar includes a 'Create new' button.

Dashboard

+ Create new

View/Edit Users

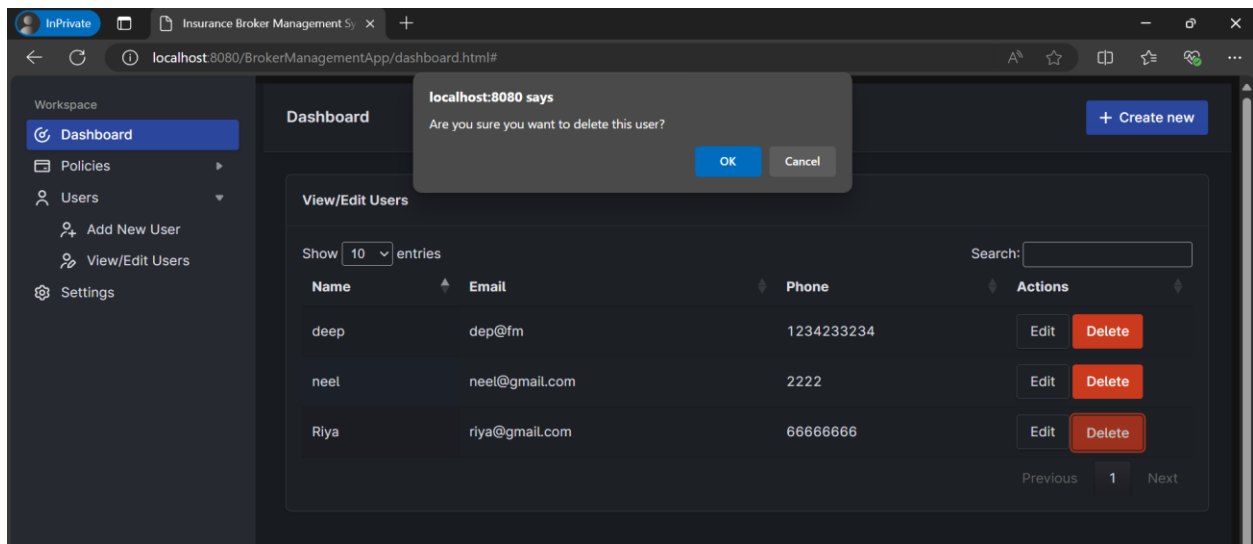
Show 10 entries

Search:

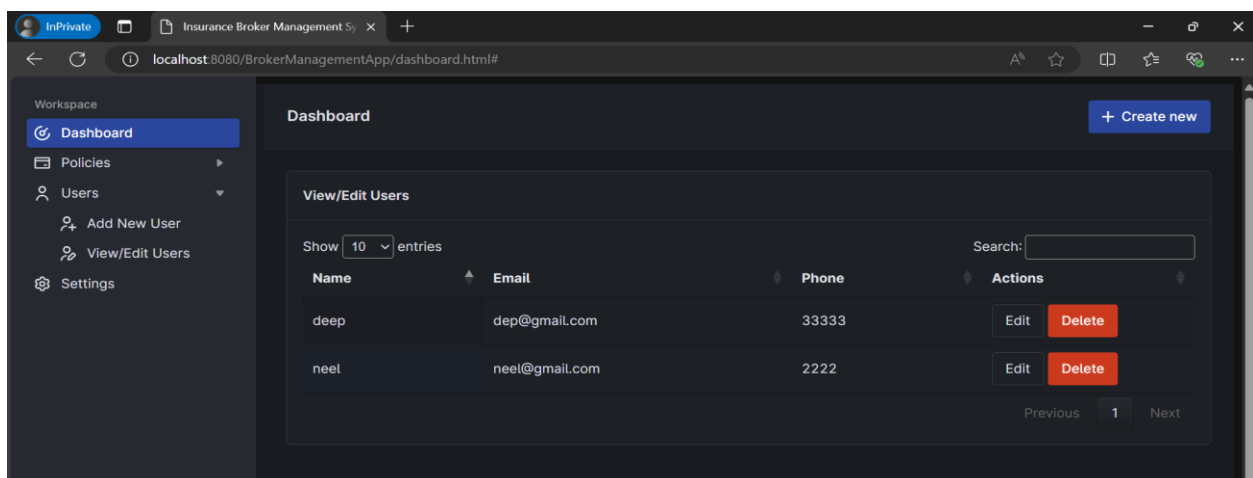
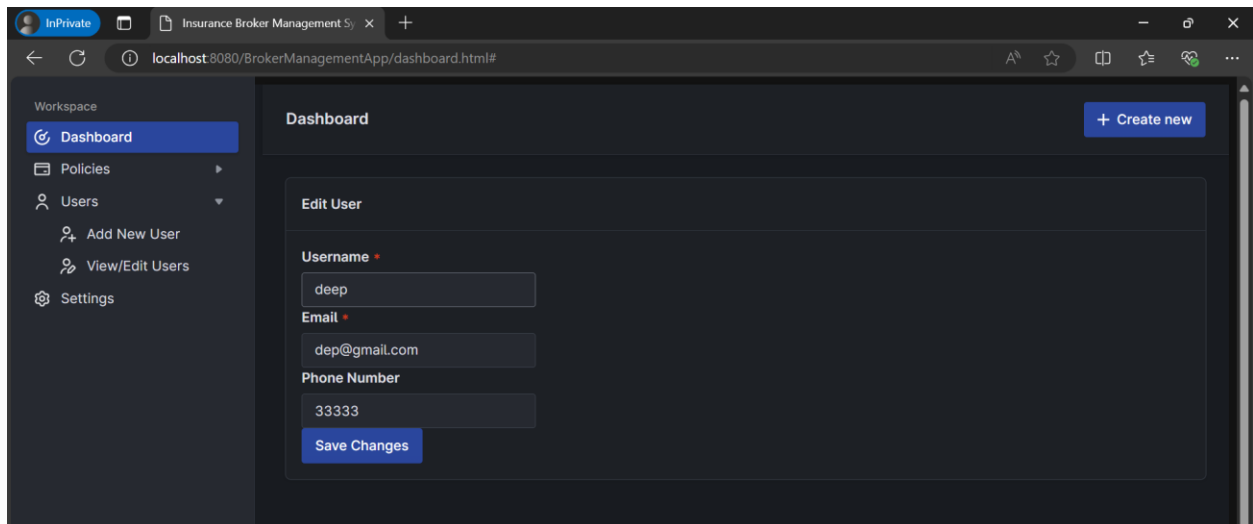
Name	Email	Phone	Actions
deep	dep@fm	1234233234	Edit Delete
neel	neel@gmail.com	2222	Edit Delete
Riya	riya@gmail.com	66666666	Edit Delete

Previous 1 Next

Delete User



Edit User



Get a Policy Quote

Get Policy Quote

localhost:8080/Broker_Assignment3/

Incognito

Downloads Google Gmail YouTube LinkedIn Cricbuzz Udemy Black Board OneDrive ChatGPT Google Maps QuillBot All Bookmarks

Policy Quote Form

Customer Name

Neel

Age

25

Policy Type

Auto

Coverage Amount

1000

Term Length (years)

1 Year

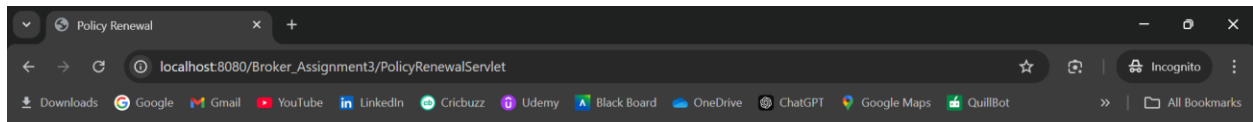
Get Quote

Policy Quote

Customer Name	Neel
Age	25
Policy Type	Auto
Coverage Amount	\$1000.0
Term Length	1 years
Premium	\$25000.0

Get Another Quote

Policy Renewal



Policy Renewal

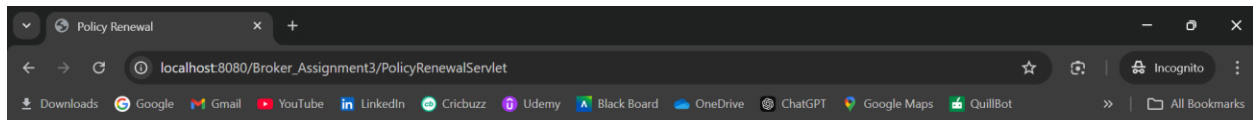
Policies

ID	Customer Name	Policy Type	Coverage	Term Length	Premium	Status	Action
1	Neel	Life	30000.0	1	5000.0	Active	☐
2	Hitesh	Health	20000.0	2	3000.0	Active	☐
3	Abdul	Auto	70000.0	1	1000.0	Expired	

New Term Length:

1 Year

Renew Policy



Policy Renewal

Policy renewed successfully!

Policies

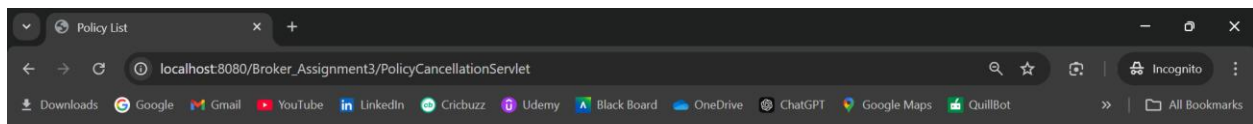
ID	Customer Name	Policy Type	Coverage	Term Length	Premium	Status	Action
1	Neel	Life	30000.0	1	5000.0	Active	☐
2	Hitesh	Health	20000.0	5	950000.0	Active	☐
3	Abdul	Auto	70000.0	1	1000.0	Expired	

New Term Length:

1 Year

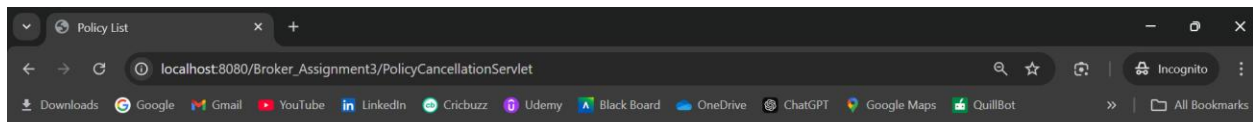
Renew Policy

Cancel Policy Feature



Policy List

ID	Customer Name	Policy Type	Coverage Amount	Term Length	Premium	Status	Cancellation Reason	Action
1	Neel	Health	30000.0	1	20000.0	Active		Cancellation Reason: <button>Cancel Policy</button>
2	Hitesh	Auto	20000.0	1	1000.0	Active		Cancellation Reason: Premium not paid <button>Cancel Policy</button>
3	Abdul	Life	10000.0	5	50000.0	Expired		

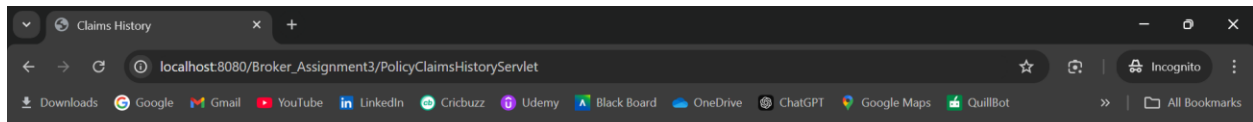


Policy List

Policy successfully canceled.

ID	Customer Name	Policy Type	Coverage Amount	Term Length	Premium	Status	Cancellation Reason	Action
1	Neel	Health	30000.0	1	20000.0	Active		Cancellation Reason: <button>Cancel Policy</button>
2	Hitesh	Auto	20000.0	1	1000.0	Canceled	Premium not paid	
3	Abdul	Life	10000.0	5	50000.0	Expired		

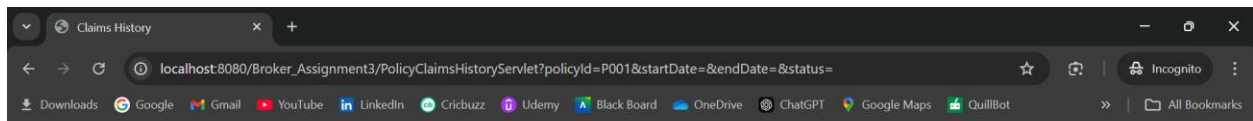
Claim History



Claims History

Policy ID: Start Date: End Date: Status:

Claim ID	Policy ID	Claim Date	Claim Amount	Status
C001	P001	2024-01-01	1300.0	Approved
C002	P001	2024-02-02	1400.0	Pending
C003	P002	2024-03-03	2600.0	Rejected
C004	P003	2024-04-04	1700.0	Approved



Claims History

Policy ID: Start Date: End Date: Status:

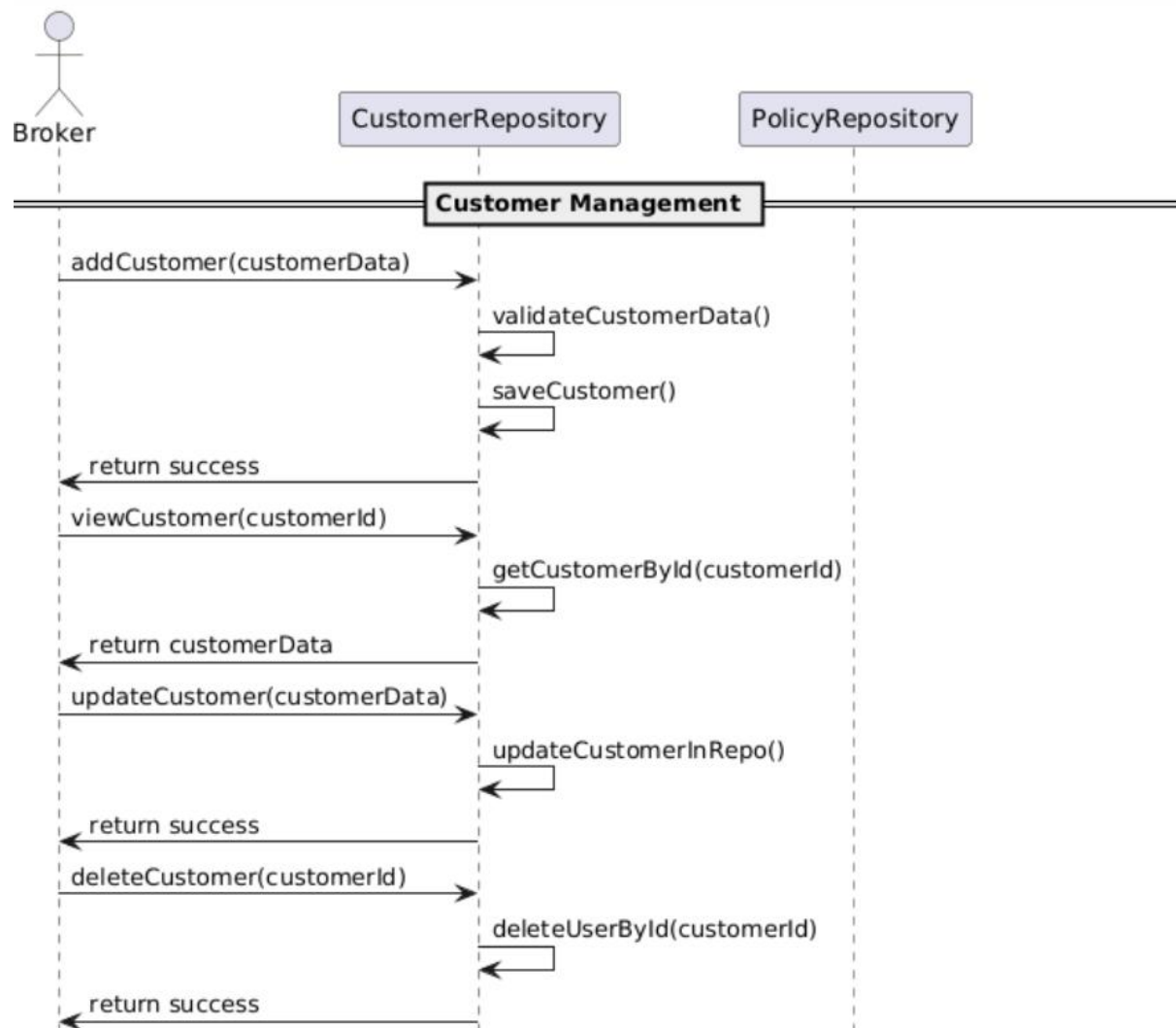
Claim ID	Policy ID	Claim Date	Claim Amount	Status
C001	P001	2024-01-01	1300.0	Approved
C002	P001	2024-02-02	1400.0	Pending

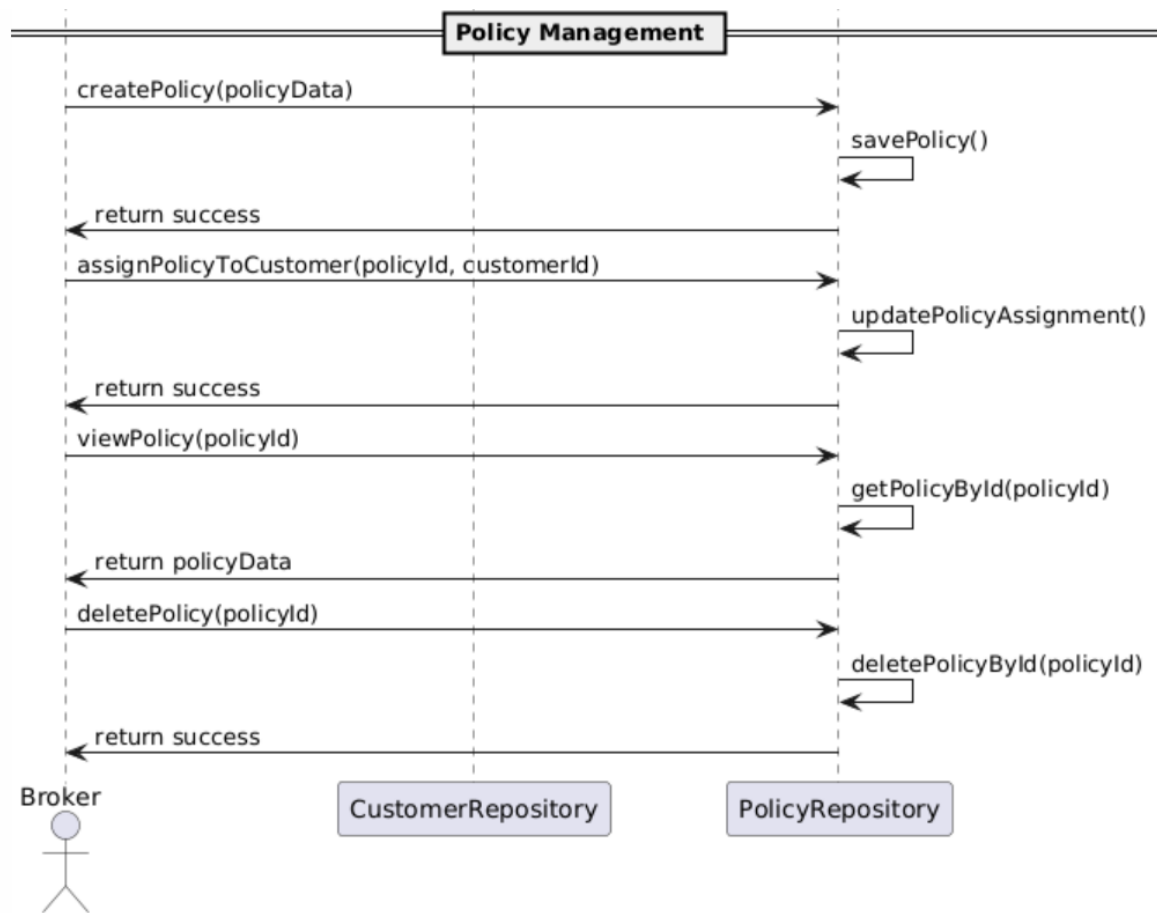
UML Diagrams

Class Diagram

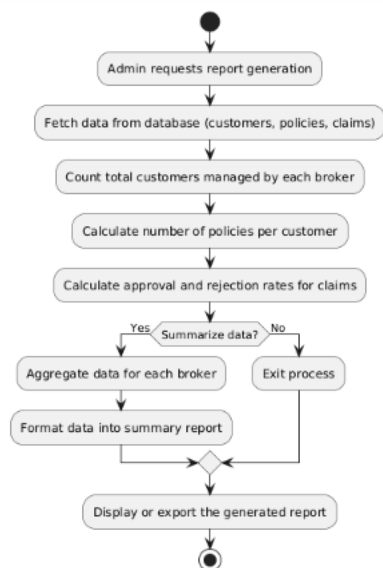


Sequence Diagram

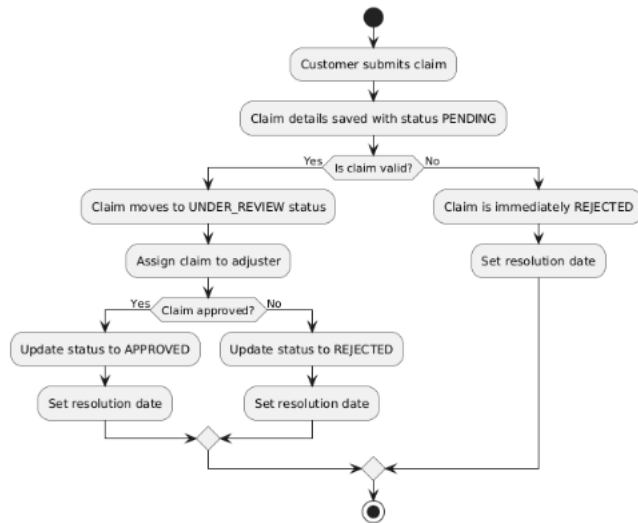




Report Activity Diagram



Policy Claim Activity Diagram



Collaboration Details

The team effectively collaborated using GitHub for version control and task management. Each team member was responsible for specific modules within the system, ensuring a clear division of responsibilities and smooth integration of the various components:

- Neel worked on report features, policy quote & claim history.
- Hitesh developed the policy management features.
- Abdul Mubeen was responsible for the front-end development.

To ensure parallel development, we adopted a feature-based branching strategy. Each team member worked on their own designated branch:

- Neel: feature/master_hibernate
- Hitesh: feature/master_hibernate
- Abdul: feature/assignment-2-front-end

This branching approach allowed us to work on individual features without interfering with others' work. Once a feature was completed, team members submitted pull requests to merge their code into the main branch. The pull request process included a thorough code review to ensure smooth integration and maintain high-quality standards. This workflow minimized conflicts, streamlined code merging, and maintained a consistent codebase.